Step 3 — The Loader and the Full Pipeline (final) — Solution

> Back to the step · Hint 1 · Hint 2

Story: this adds the document loader (Doc08's pattern), the pipeline function that ties load → extract → validate together into one call, `main.py`, and the integration test against a real model. **If not:** without a real loader the project would only work on text you pasted in by hand — real invoices arrive as files, and often as PDFs.

Sample invoices (`samples/`)

```
# samples/clean.txt
Vendor: Acme Supplies
Invoice Number: INV-1001
Items:
  3 x Widget @ 10.00
  5 x Bolt @ 2.00
Total: 40.00
```

```
# samples/bad_total.txt
Vendor: Acme Supplies
Invoice Number: INV-1002
Items:
  3 x Widget @ 10.00
Total: 99.00
```

```
# samples/missing_number.txt
Vendor: Acme Supplies
Items:
  2 x Cable @ 5.00
Total: 10.00
```

The loader

```
# src/loader.py
# Read an invoice file into plain text. .txt/.md are read directly; .pdf uses
# a document loader (Doc08). The model only ever sees this text.
from src.exceptions import UnsupportedFileError


def load_invoice_text(path):
    lower = path.lower()
    if lower.endswith((".txt", ".md")):
        with open(path) as invoice_file:
            return invoice_file.read()
    if lower.endswith(".pdf"):
        # needs: pip install pypdf
        from langchain_community.document_loaders import PyPDFLoader
        pages = PyPDFLoader(path).load()
        texts = []
        for page in pages:
            texts.append(page.page_content)
        return "\n".join(texts)
    raise UnsupportedFileError(f"Unsupported invoice file type: {path}")
```

The full pipeline

Finish `src/pipeline.py` by adding the `Result` holder and `process_invoice` (the `extract_invoice` from Step 2 stays):

```
# src/pipeline.py  (final — add to the Step 2 version)
from dataclasses import dataclass

from src.loader import load_invoice_text
from src.utils.logger import get_logger
from src.validator import validate_invoice


@dataclass
class Result:
    invoice: Invoice
    problems: list


def process_invoice(model, path):
    logger = get_logger(__name__)
    text = load_invoice_text(path)
    invoice = extract_invoice(model, text)
```

```
    problems = validate_invoice(invoice)
    logger.info(f"processed {path}: {len(problems)} problem(s)")
    return Result(invoice=invoice, problems=problems)
```

main.py

```
# main.py
import sys

from src.config import load_config
from src.models.llm import get_model
from src.pipeline import process_invoice


def main():
    if len(sys.argv) < 2:
        print("usage: python main.py <invoice-file>")
        return
    path = sys.argv[1]
    config = load_config()
    model = get_model(config)
    result = process_invoice(model, path)
    invoice = result.invoice

    print(f"\nInvoiceIQ result for: {path}")
    print(f"  vendor:         {invoice.vendor}")
    print(f"  invoice_number: {invoice.invoice_number}")
    print(f"  line items:     {len(invoice.line_items)}")
    print(f"  stated total:   {invoice.total}")

    if not result.problems:
        print("\nCleared: no problems found.")
    else:
        print("\nProblems:")
        for problem in result.problems:
            print(f"  - {problem}")


if __name__ == "__main__":
    main()
```

The integration test (real model)

```
# tests/integration/test_live_extract.py
# Real model. Run from the repo root:
#   python -m tests.integration.test_live_extract
from src.config import load_config
from src.models.llm import get_model
from src.pipeline import process_invoice


def test_clean_invoice_clears():
    config = load_config()
    model = get_model(config)
    result = process_invoice(model, "samples/clean.txt")
    ok = result.invoice.invoice_number == "INV-1001" and not result.problems
    result1 = "WRONG"
    if ok:
        result1 = "OK"
    print(f"case 1 (clean invoice clears)     -> {result1}")


def test_bad_total_is_flagged():
    config = load_config()
    model = get_model(config)
    result = process_invoice(model, "samples/bad_total.txt")
    flagged = False
    for problem in result.problems:
        if "total mismatch" in problem:
            flagged = True
    status = "OK"
    if not flagged:
        status = "WRONG"
    print(f"case 2 (bad total is flagged)     -> {status}")


if __name__ == "__main__":
    test_clean_invoice_clears()
    test_bad_total_is_flagged()
```

Expected run:

```
$ python main.py samples/bad_total.txt

InvoiceIQ result for: samples/bad_total.txt
  vendor:         Acme Supplies
  invoice_number: INV-1002
  line items:     1
  stated total:   99.0
```

```
Problems:
  - total mismatch: line items sum to 30.0, invoice says 99.0
```

That's the whole project: the model read a messy file into a clean `Invoice`, and your deterministic validator caught the total that didn't add up — before anyone posted it. Swap `PyPDFLoader` in and the exact same pipeline handles PDFs.